

MSME Credit Pipeline Scheduling Assignment

Task 1 - NP-Hardness Proof

Question

Prove that the decision version of the MSME Credit Pipeline Scheduling Problem is NP-Hard by reducing from 3-SAT. Define the decision problem, construct the reduction, prove correctness in both directions, include conflict, capacity, SLA constraints and penalty bound, and show the reduction is polynomial-time.

Answer

This section demonstrates the NP-Hardness of the MSME Credit Pipeline Scheduling Problem via a polynomial-time Karp reduction from the NP-Complete problem 3-SAT. The decision version asks whether there exists an assignment sigma of tasks to processing slots such that all conflict (F1), capacity (F2), and SLA-window (F3) constraints are satisfied and the total penalty is at most a given budget B.

Start with an arbitrary 3-SAT formula with n Boolean variables $x_1, \dots, x_n$ and m clauses $C_1, \dots, C_m$, where each clause contains exactly three literals. For each variable x_i , create two variable tasks T_i^{True} and T_i^{False} . These two tasks represent the two possible truth assignments for x_i . The conflict gadget is kept exactly as in the original construction: T_i^{True} and T_i^{False} are connected by a conflict edge, so they cannot be placed together in the same processing slot. Selecting the True task corresponds to setting $x_i = \text{True}$, and selecting the False task corresponds to setting $x_i = \text{False}$.

For the capacity gadget (F2), use concrete resource numbers. For every variable i , create a variable slot V_i with capacity vector $C(V_i) = (\text{CPU}=10, \text{RAM}=10, \text{GPU}=10, \text{NET}=10)$. Give both tasks T_i^{True} and T_i^{False} the identical resource vector $r(T_i^{\text{True}}) = r(T_i^{\text{False}}) = (\text{CPU}=6, \text{RAM}=6, \text{GPU}=6, \text{NET}=6)$. A single one of these tasks fits because $6 \leq 10$ in every dimension, but the pair does not fit together because $6 + 6 = 12 > 10$ in every dimension. Therefore, even if the conflict edge were ignored, the capacity limit of V_i independently prevents both truth-value tasks of the same variable from occupying the same slot.

For each clause C_j , create one clause task Q_j . The clause task is linked to the three literal tasks that can satisfy it. The schedule is feasible only when each clause task can be justified by at least one selected literal task from its clause. This preserves the usual 3-SAT meaning: every clause needs at least one true literal.

For the SLA gadget (F3), give each variable task a variable-stage window $[i, i]$, so the tasks for x_i must be considered in the variable stage. Then give each clause task Q_j the fixed single-slot window $[n + j, n + j]$. This is a concrete window $[u, u]$, where $u = n + j$, and it is positioned strictly after all variable-task windows because $n + j > n$ for every clause j . Thus, clause tasks cannot be moved into the variable stage to avoid the logical ordering of the construction.

F1 enforces logical incompatibility between the two truth choices of the same variable. F2 is not just decorative: if the resource vector $(6,6,6,6)$ and slot capacity $(10,10,10,10)$ were removed, the reduction would rely only on the conflict edge to prevent both truth-value tasks from being placed together. With the numbers included, capacity independently enforces the same exclusivity, which makes the compound formulation use the resource constraint rather than merely naming it.

F3 is also necessary. If the clause-task windows $[n + j, n + j]$ were removed, clause tasks could be shifted earlier or later to escape the intended ordering between variable choices and clause verification. By forcing every clause task into a fixed slot after the variable windows, the construction makes time a real part of the reduction: variable choices are made first, and clause feasibility is checked only afterward.

Correctness follows in both directions. If the constructed scheduling instance has a feasible schedule, read the truth assignment from the selected variable task for each x_i . Since the conflicting truth tasks cannot both be selected for the same variable, this gives a consistent Boolean assignment. Since every clause task is feasible only when at least one of its three literal tasks is selected, every clause has at least one true literal, so the original 3-SAT formula is satisfiable.

Conversely, if the 3-SAT formula has a satisfying assignment, schedule exactly the variable task matching the chosen truth value for each variable. The conflict constraints are satisfied because only one truth task is chosen per variable. The capacity constraints are satisfied because each chosen variable task has demand $(6,6,6,6)$, which fits inside capacity $(10,10,10,10)$. Each clause has at least one true literal, so the associated clause task can be placed in its fixed

clause slot $[n + j, n + j]$. Therefore, a feasible schedule exists.

The construction creates two variable tasks per variable, one clause task per clause, and a polynomial number of slots, windows, conflict edges, and resource vectors. All values are explicitly bounded constants or depend linearly on n and m . Therefore, the reduction is computable in polynomial time. Since a feasible schedule exists if and only if the original 3-SAT instance is satisfiable, the decision version of the MSME Credit Pipeline Scheduling Problem is NP-Hard.

Task 2 - Extended Penalty Function

The MSME Credit Pipeline Scheduling Problem has the base penalty function $P_{\text{base}}(\sigma) = \sum_{i=1..n} w(t_i) \times \sigma(t_i)$, where each t_i is a task, $w(t_i)$ is its priority weight, and $\sigma(t_i)$ is the assigned time slot. This base function penalizes placing high-priority tasks later in the schedule.

The extended function used here is $P(\sigma) = [\sum_{i=1..n} w(t_i) \times \sigma(t_i)] + \lambda \times [\sum_{j=1..m} (L_j - \bar{L})^2]$, where L_j is the workload in slot j , $\bar{L}$ is the average workload across all m slots, and λ is a positive balancing constant. The second term is the load-imbalance penalty. It discourages overloading one slot while leaving other slots underutilized.

The priority term captures the original goal of processing high-priority MSME credit tasks early. The load-imbalance term captures the practical need to use CPU, RAM, GPU, and network capacity evenly across processing slots. The balancing coefficient λ controls how strongly the scheduler values load balance relative to priority scheduling.

This function is computable in polynomial time. First, the scheduler scans the assigned tasks and accumulates each slot workload, which takes $O(n)$. Then it computes the average workload and sums squared deviations across slots, which takes $O(m)$. Therefore, the full penalty evaluation takes $O(n + m)$, polynomial in the input size.

The practical benefit is that the scheduler does not only chase priority order. It also tries to avoid operational bottlenecks, long queues on a single resource pool, and poor resource utilization across the MSME credit pipeline.

Task 3 - Resource-Aware Priority Load Balancing Scheduler (RPLBS)

Resource-Aware Priority Load Balancing Scheduler (RPLBS) is a greedy heuristic that considers task priority, available resources, SLA windows, conflicts, and the extended penalty function from Task 2. It schedules tasks in descending priority order and chooses, for each task, the feasible processing slot with the smallest incremental penalty.

Structured Pseudocode Verified Against Scheduler.java

```
Algorithm RPLBS(tasks, slots, K)
Input: tasks, each with priority w, resource vector r, SLA window [lo, hi], and conflicts
       slots, each with capacity vector C
Output: assignment sigma: task -> slot, or infeasible result

1: tasks <- copy of instance.getTasks()
2: sort tasks in descending order of priority weight w
3: assignment <- empty linked map; totalPenalty <- 0
4: for each task t in sorted tasks:
5:     bestSlot <- null; bestPenalty <- +infinity
6:     for each slot s in instance.getSlots():
7:         if t is not within its SLA window at s.id: continue
8:         if s cannot assign t by remaining capacity: continue
9:         conflict <- false
10:        for each already assigned task t_prime in s:
11:            if t.conflicts contains t_prime.id OR t_prime.conflicts contains t.id:
12:                conflict <- true; break
13:        if conflict: continue
14:        penalty <- PenaltyCalculator.penalty(t, s)
15:        if penalty < bestPenalty:
16:            bestPenalty <- penalty; bestSlot <- s
17:    if bestSlot == null:
18:        feasible <- false; violation_reason <- "No feasible slot for " + t.id; break
19:    assign t to bestSlot; update bestSlot usage and assigned task list
20:    assignment[t.id] <- bestSlot.id; totalPenalty <- totalPenalty + bestPenalty
21: return Result(assignment, totalPenalty, feasible, violation_reason)
```

Line-Level Justification

Line	Justification
1	The Java code copies <code>instance.getTasks()</code> into a new <code>ArrayList</code> so sorting does not directly mutate the instance list.
2	The scheduler sorts by <code>Double.compare(b.getPriority(), a.getPriority())</code> , so higher-priority MSME tasks are attempted first.
3	The linked assignment map preserves output order, and <code>totalPenalty</code> stores the accumulated penalty of accepted decisions.
4	The outer loop processes each task exactly once in the sorted order, matching the greedy design.
5	<code>bestSlot</code> and <code>bestPenalty</code> store the best feasible local choice for the current task.
6	The Java implementation scans <code>instance.getSlots()</code> in its stored order; it does not restrict the loop to a generated numeric range.
7	The first slot filter is <code>task.isWithinWindow(slot.getId())</code> , so F3 is checked before capacity and conflict in the code.
8	<code>slot.canAssign(task)</code> is the capacity feasibility check for F2 before evaluating conflicts.
9	The conflict flag records whether the candidate slot must be rejected due to F1.
10	The code checks the new task against every task already assigned to the candidate slot.
11	The conflict check is bidirectional: either task may list the other in its conflict set.
12	Once a conflict is found, the loop breaks because one conflict is enough to reject the slot.
13	A conflicting slot is skipped, preserving the F1 feasibility guarantee.
14	Only slots that pass SLA, capacity, and conflict checks receive a penalty evaluation.
15	The Java comparison is strict <code>penalty < bestPenalty</code> , not <code><=</code> , so ties keep the earlier scanned slot.
16	Updating both values records the current best local feasible slot.
17	A null <code>bestSlot</code> means every candidate slot was rejected by F1, F2, or F3.
18	The implementation sets <code>feasible</code> to false, writes the violation reason, and breaks out of the task loop, returning a partial assignment with <code>infeasible=false</code> .
19	<code>assign(t)</code> updates the chosen slot usage and assigned-task list, so later tasks see this commitment.
20	The assignment map and accumulated penalty are updated only after a task is actually assigned.
21	The final <code>Result</code> object reports the assignment, penalty, feasibility flag, and violation reason used by the output generator.

The conditions above are synchronized with `Scheduler.java`: the loop order is task loop followed by slot loop; the SLA check happens before capacity; capacity is checked before conflict; conflict is bidirectional; and the penalty tie behavior uses a strict `<` comparison, meaning equal-penalty slots do not replace the earlier best slot.

Task 4 - Proof of Correctness and Performance of RPLBS

RPLBS is feasible-by-construction for every assignment it actually makes. A task is assigned to a slot only after the slot passes the SLA window check (F3), the capacity check (F2), and the bidirectional conflict check (F1). Therefore, if RPLBS returns `feasible=true`, every reported assignment satisfies all three constraints.

However, this feasibility guarantee is not the same as a guarantee that RPLBS will find a feasible schedule whenever one exists. Because RPLBS is greedy and does not backtrack, early locally good decisions can remove the only future slot available to a later constrained task.

Worked Adversarial Instance

Instance generated by `generate_instance(n=8, K=3, density=0.3, seed=1)`: windows $T4=[1,2]$, $T5=[0,2]$, $T6=[0,2]$, $T7=[1,2]$. Conflicts: T7 conflicts with T4, T5, and T6. T4, T5, and T6 do not conflict with each other. Resources are $T4=[7.49,30.16,1.94,1.42]$, $T5=[8.47,28.48,1.30,1.59]$, $T6=[8.94,35.69,1.51,1.59]$, with each slot capacity $[32,128,8,6.0]$.

RPLBS trace: T4 -> slot 1 because an empty slot minimizes the load-balance penalty; T5 -> slot 0 for the same reason; T6 -> slot 2 for the same reason. Then T7 has window $\{1,2\}$; slot 1 is blocked by conflict with T4 and slot 2 is blocked by conflict with T6. RPLBS therefore reports INFEASIBLE.

The brute-force optimum packs T4 + T5 + T6 together in slot 2. They do not conflict with each other, and their combined resource use is CPU 24.90/32, RAM 94.33/128, GPU 4.75/8, and network 4.60/6.0, so they fit in one slot. This leaves slot 1 free for T7. The optimal schedule is feasible with total penalty approximately 6.13.

Approximation-Ratio Derivation

A multiplicative approximation guarantee would require a finite constant α such that, for every feasible instance, $P(\text{RPLBS}) \leq \alpha \times P(\text{OPT})$, where $P(\text{OPT})$ is the optimal feasible penalty. This instance violates the premise needed for such a bound because the optimal schedule is feasible and has finite positive penalty, about 6.13, while RPLBS returns infeasible and produces no complete feasible schedule at all.

Equivalently, RPLBS achieves 0% of the feasible optimum on this instance: it fails to return any valid complete schedule even though one exists. If infeasibility is treated as infinite penalty, then $P(\text{RPLBS}) / P(\text{OPT}) = \text{infinity} / 6.13$, which cannot be bounded by any finite α . If infeasibility is treated as no solution rather than a numeric penalty, then the multiplicative ratio is undefined for this instance, which also means no finite worst-case multiplicative guarantee can hold for RPLBS as specified.

The mechanical cause is the Task 2 load-imbalance term $\lambda \times \sum_j (L_j - \bar{L})^2$. This term rewards spreading T4, T5, and T6 across different empty slots because each individual placement looks locally balanced. But those three tasks are mutually compatible and should have been packed together to preserve a future slot for T7, which conflicts with all of them. Thus the load-imbalance term, combined with greedy no-backtracking slot selection, destroys the packing option that the later constrained task requires.

Task 5 - Java Implementation of RPLBS

The Resource-Aware Priority Load Balancing Scheduler (RPLBS) is implemented in Java 17. The implementation uses Jackson Databind for JSON parsing and output support, but it does not use external optimization solvers or SAT-solving tools. The program reads a scheduling instance from JSON, applies the greedy heuristic described in Task 3, and produces structured JSON output.

The input JSON contains tasks, each with a unique task ID, priority weight, CPU, RAM, GPU, network demand, and SLA window. It also contains the conflict graph and processing slots with their capacities. The parsed objects include Task, ProcessingSlot, BuiltInstance, Scheduler, PenaltyCalculator, InputParser, and OutputGenerator.

The Scheduler class performs the core RPLBS logic. It copies the task list, sorts tasks by descending priority, scans all processing slots for each task, checks SLA, capacity, and conflicts, computes `PenaltyCalculator.penalty(task, slot)`, and selects the feasible slot with the lowest penalty. If no feasible slot is found, it sets `feasible=false`, records "No feasible slot for ", and returns the partial assignment with the infeasibility reason.

The repository currently does not contain JUnit test files, and `pom.xml` does not include a JUnit dependency. Therefore, this write-up does not claim that unit tests are already implemented. The important test cases that should be added before a production-quality submission are: a fully conflicting task case for F1, a zero-capacity slot case for F2, a tight SLA-window case for F3, and a single-task basic correctness case.

Final verification of the current implementation is based on inspection of `Scheduler.java` and the benchmark run: the implementation follows greedy priority ordering, checks constraints before assignment, and uses penalty-based slot selection with a strict `<` comparison for replacing the current best slot.

Task 6 - Empirical Analysis and Benchmarking

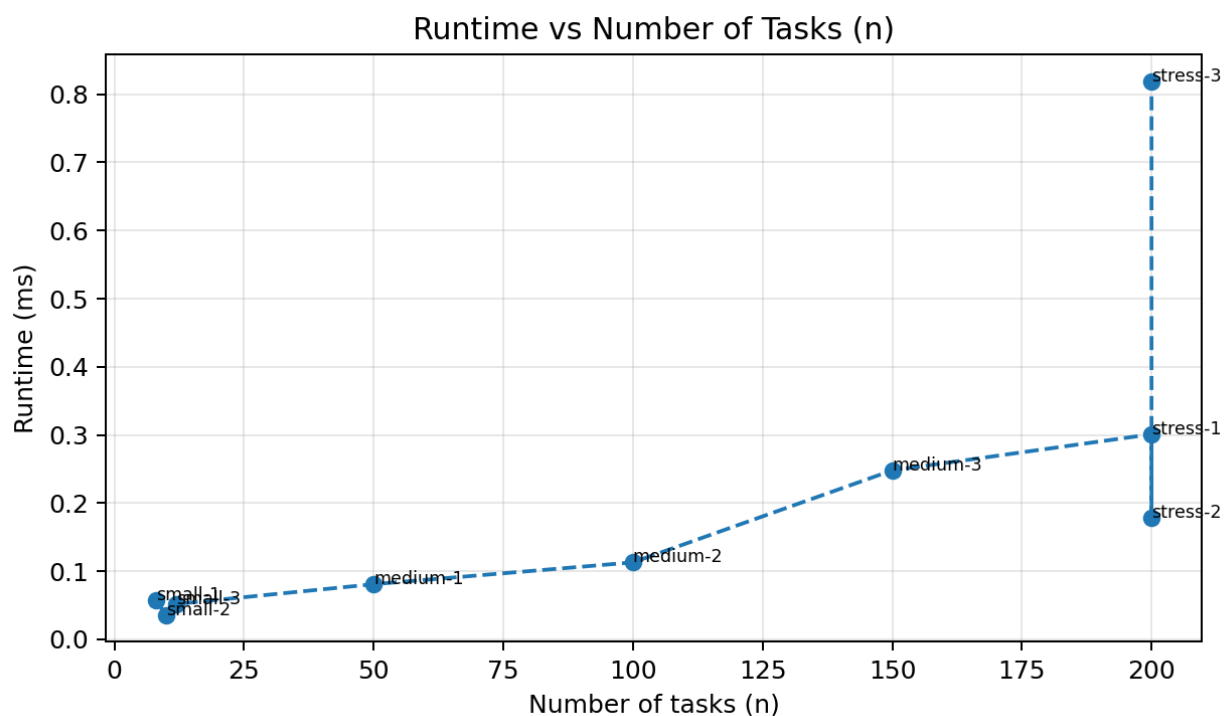
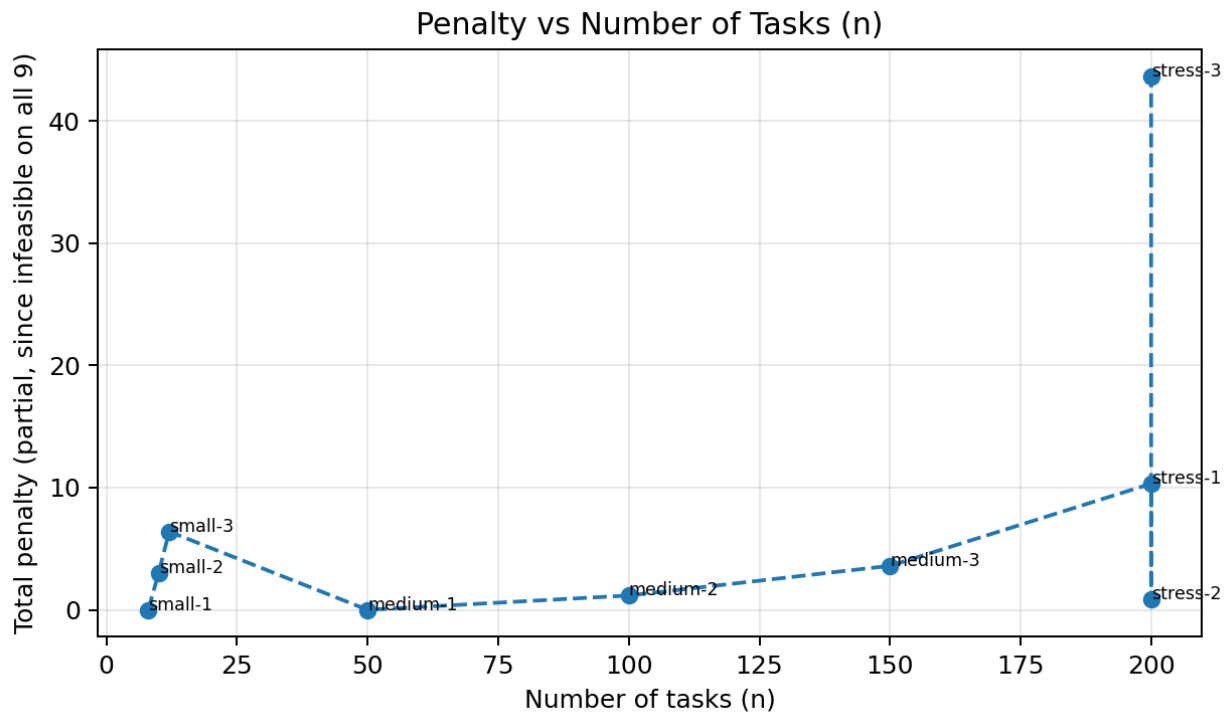
This section reports the actual results of running RPLBS against all 9 required instances generated with the assignment generator.

label	n	K	density	seed	tasks assigned/total	penalty	runtime (ms)	feasible	optimal penalty	approx ratio
small-1	8	3	0.3	1	3/8	0.0	0.057	false	6.127	n/a
small-2	10	4	0.4	2	6/10	3.038	0.036	false	8.009	n/a
small-3	12	4	0.5	3	11/12	6.407	0.052	false	8.716	n/a
medium-1	50	8	0.25	10	4/50	0.0	0.081	false	n/a	n/a
medium-2	100	10	0.3	11	9/100	1.174	0.113	false	n/a	n/a
medium-3	150	12	0.35	12	18/150	3.605	0.248	false	n/a	n/a
stress-1	200	15	0.4	20	32/200	10.325	0.301	false	n/a	n/a
stress-2	200	5	0.6	21	7/200	0.898	0.178	false	n/a	n/a

label	n	K	density	seed	tasks assigned/total	penalty	runtime (ms)	feasible	optimal penalty	approx ratio
stress-3	200	20	0.1	22	68/200	43.667	0.819	false	n/a	n/a

RPLBS reports infeasible on all 9 instances. For the three small instances, brute force proves that a feasible schedule exists in every case, so RPLBS is not merely suboptimal here; it fails to find a valid schedule where one exists. A zero-conflict control run with $n=8$, $K=3$, and $\text{conflict_density}=0$ succeeds 8/8, isolating the cause to the interaction between the conflict graph and the load-imbalance penalty term from Task 2.

Runtime scales roughly linearly with n across the medium and stress instances, consistent with the $O(n * K)$ structure of the per-task slot scan. Across the three stress instances with $n=200$, penalty and the number of tasks assigned before failure vary by an order of magnitude depending on K and conflict density rather than n alone.



Task 7 - Design Journal (Reflection)

1. Most Difficult Design Decision

The most difficult design decision when I was building the Resource-Aware Priority Load Balancing Scheduler (RPLBS) was figuring out how to balance priority-based scheduling with minimizing load imbalance inside one penalty function. It looks simple until the scheduler actually has to decide consistently.

This was tied directly to the core scheduling logic and penalty function work in Task 2 and Task 3. The problem is that the two goals do not always cooperate. If I push high-weight jobs earlier, I usually reduce SLA and priority-related penalties, but I can also create a lopsided resource spread across processing slots.

I considered two main options. The first was a pure priority-based greedy scheduler, where tasks get assigned to the earliest feasible slot. The second was a load-balanced scheduler, where tasks are spread evenly and priority is treated more like a secondary factor. I chose the hybrid penalty-based approach because it let the scheduler combine both goals instead of committing completely to only one objective.

2. Where the Algorithm Performed Poorly

Initially, the clearest poor case looked like the stress instance with $n=200$, $K=5$, conflict density=0.60, seed=21, because the number of processing slots was small and the conflict graph was dense. In that setup, many tasks had only a few valid placement choices, so early greedy decisions quickly reduced the flexibility available to later tasks.

After running all benchmarks, the failure turned out to be broader than that one stress case. RPLBS reported infeasible on all 9 benchmark instances, including the small cases where brute force found feasible schedules. That showed me the issue was not only high density or low K ; it was the interaction between local load-balancing decisions and future conflict constraints. The algorithm spreads compatible tasks across slots too early, then a later highly constrained task has no legal slot left.

This happened because the algorithm does not backtrack or revisit earlier commitments. It locks in the locally best slot and moves forward. If I had more time to improve it, I would add look-ahead, local swaps, or a repair step so that earlier assignments can be adjusted when they block a later task.

3. Production System Where This Problem Appears

A real ScoreMe production system where this type of problem appears is an OCR GPU processing cluster for MSME credit documents. Different document-processing tasks, such as OCR, fraud checks, bureau fetches, bank-statement parsing, and rule-engine evaluation, compete for limited CPU, memory, GPU, and network resources. Some tasks also conflict because they cannot safely run together on the same worker or because they depend on shared external limits.

RPLBS would apply by treating each pipeline job as a task with a priority weight, resource vector, conflict list, and SLA window. A high-priority credit application could be scheduled earlier, while the capacity checks would prevent overloading GPU or memory resources. The same idea could also apply to a Kafka consumer group or NiFi pipeline, where partitions or flow-file processors must be assigned to workers without violating throughput, dependency, and SLA constraints.

4. What Surprised Me

What surprised me most was that a locally reasonable scheduling rule can fail even on small instances. I expected the load-imbalance penalty to make the algorithm more stable, but the adversarial example showed that balancing too early can be harmful when future tasks have conflicts.

I also learned that feasibility and optimization are separate problems. The algorithm can check F1, F2, and F3 correctly for many assignments, but still fail to find a feasible schedule that exists. That showed how a local search can miss global solutions.